



RUBI: Relative-Urgency Based Initialization for Solving Job Shop Scheduling Problem

Jiwon Baek¹ , Hyunjin Oh¹ , and Jong Hun Woo^{1,2} 

¹ Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul 08826, South Korea
baekjiwon@snu.ac.kr

² Research Institute of Marine Systems Engineering, Seoul National University,
1 Gwanak-ro, Gwanak-gu, Seoul 08826, South Korea

Abstract. This paper introduces a new initialization scheme for enumerative algorithms in the Job Shop Scheduling Problem (JSSP). JSSP is a well-known NP-hard optimization problem that models one of the most general forms of manufacturing systems and has been extensively studied in the field of operations research. The proposed method, named Relative-Urgency Based Initialization (RUBI), improves the search efficiency of evolutionary algorithms for makespan optimization. The foundation of our approach is the Relative-Urgency (RU) principle, which theoretically aims to minimize the lower bound of the makespan. To quantify how well a JSSP solution aligns with this principle, we introduce the RU score—a metric that measures the degree to which the solution follows the RU principle. Empirical analysis reveals a strong correlation between the RU score and makespan. Based on this, we demonstrate that RUBI effectively guides the search process toward sub-optimal regions that are closer to the global optimum. Experimental results show that RUBI consistently improves the performance of the standard Genetic Algorithm (GA), both in terms of solution quality and time to reach the optimum. These findings suggest that the RU principle can contribute to the development of more efficient heuristics and frameworks for solving JSSP.

Keywords: Job Shop Scheduling Problem · Genetic Algorithm · Manufacturing Optimization

1 Introduction

The Job Shop Scheduling Problem (JSSP) is a combinatorial optimization problem that effectively models one of the most generalized forms of modern manufacturing systems. In an $N \times M$ JSSP, there are N jobs, each consisting of M operations with predefined precedence constraints. Each operation must be processed on a specific machine. The objective of JSSP is to minimize the makespan, or the completion time of all jobs. Other objectives, such as minimizing tardiness, can also be considered. Still, minimizing makespan is one of the most important objectives, as reducing the total time consumed is critical in terms of both cost efficiency and productivity.

Various methodologies have been developed to tackle JSSP, each with distinct objectives. However, before applying specific approaches, it is crucial to understand the inherent structure of the JSSP. This can enhance the effectiveness of different approaches by providing insights into estimating lower bounds, designing efficient heuristics and search techniques, and developing feature representations for AI-based methods. In this study, we propose a simple yet intuitive idea for better understanding JSSP. Imagine a queue where multiple people are waiting—wouldn't it be reasonable to prioritize the person whose task is most urgent, i.e., the one with the most remaining steps?

Inspired by this intuition, we introduce the principle of *Relative Urgency* (RU), which prioritizes jobs based on the number of remaining operations to minimize makespan. We also propose the Relative-Urgency Based Initialization (RUBI) scheme for search techniques such as Genetic Algorithms (GA). Although the RU principle was initially derived from rough assumptions, its practical effectiveness was validated through statistical analysis. Tested on various JSSP instances, RUBI demonstrated its effectiveness in terms of both optimality and execution time, proving its capabilities as a new initialization technique of enumerative algorithms for the optimization of JSSP.

2 Literature Review

The Job Shop Scheduling Problem (JSSP) has been extensively studied in the field of optimization due to its combinatorial complexity. Various constraints, including precedence relationships between operations and machine-operation compatibility, make JSSP a highly complex, NP-hard problem. Early research on JSSP focused on exact methods, such as integer programming and disjunctive graph-based formulations, to compute optimal solutions or tighten lower bounds. The first mathematical formulation of scheduling problems using Integer Programming (IP) dates back to [29], and the IP model of JSSP were formulated in [19]. Many studies have since explored mathematical formulations to either compute exact solutions or improve lower bounds of JSSP instances. Related works include [6], where a two-job relaxation was introduced to tighten lower bounds, and [14], where Lagrangian relaxation was utilized. A time-oriented branching scheme was suggested by [20], improving the branch-and-bound algorithm's lower bounding procedure.

Despite these advancements, exact methods become computationally impractical for large JSSP instances, leading to the development of approximation methods. Heuristic methods aim to find near-optimal solutions without guaranteeing optimality. In [8], a priority rule based on slack time was introduced, and [2] introduced the Shifting Bottleneck Heuristic (SBH), which was later extended by [25]. Constraint-based heuristics [9] and maintenance-aware heuristics [30] further enhanced practical relevance.

Metaheuristics systematically explore the search space without domain-specific knowledge, making them highly adaptable to various combinatorial optimization problems. Among various metaheuristic techniques, Genetic Algorithms (GA) have been widely studied due to their adaptability [12, 21, 22, 35]. A key advancement was the introduction of repetitive permutation encoding, where each job appears exactly M times in a chromosome [4, 8]. This scheme became a widely accepted practice and was later adopted in [23, 26, 27, 31, 33]. Hybrid GAs, such as those incorporating local search

or Tabu Search, have also been proposed [13, 24]. Other metaheuristics such as Simulated Annealing [18, 28], Cuckoo Search [3], Artificial Bee Colony [4], and Grey Wolf Optimization [16] have demonstrated competitive performance.

Recently, deep reinforcement learning (DRL) has emerged as a promising paradigm for JSSP, with models representing the problem as a Markov Decision Process (MDP) and utilizing deep neural networks (DNNs) to learn scheduling policies [15, 34]. DRL approaches effectively extract features from complex job-shop structures and have shown promising results in approximating optimal solutions.

Although the specific approaches may differ, obtaining well-constructed feasible solutions is crucial in JSSP research. These near-optimal solutions can guide the search process toward optimality or serve as strong baselines during solution construction. Many studies utilize classical dispatching rules such as Shortest Processing Time (SPT), Longest Processing Time (LPT), First-In-First-Out (FIFO), Most Work Remaining (MWR), or Critical Ratio (CR). While dispatching heuristics are effective in generating non-delay schedules, they do not guarantee optimality. For example, in some optimal schedules, it is observed that a machine remains idle for a moment to wait for a job that has not yet arrived in order to reduce the overall makespan. Consequently, a variety of initialization methods, or solution-construction methods were studied. These include the Fuzzy Priority Rule combining SPT and CR [7], and a priority-based construction and improvement algorithm using shortest remaining time [1]. In [32], the Hopfield Neural Network model was enhanced to solve JSSPs formulated as integer linear programs, utilizing the neural network structure to search for optimal integer solutions. A composite rule-based initialization strategy for GA was introduced in [17], selecting jobs according to weighted priority scores from various dispatching rules.

It is worth noting that solutions are often represented as sequences of jobs, or operations, indicating their processing order. A widely used initialization strategy that adopts this representation is the Giffler & Thompson (GT) algorithm. Originally proposed in [11], the algorithm sequentially schedules operations by selecting, at each step, the operation with the earliest finish time. This simple yet effective method for generating non-delay schedule has become a widely accepted practice in the field. Our study also suggests a solution construction scheme that can be used as an initial solution of JSSP. Its effectiveness will be evaluated in comparison with other dispatching-rule-based approaches as well as the GT algorithm.

3 Problem Definition

In the classical JSSP, there exist N jobs and M machines. Each job consists of M operations, subject to precedence constraints. The following are the typical constraints of the JSSP:

- Each machine can process only one operation at a time, and no preemption is allowed.
- The operations of each job must be processed in a predefined order.
- Each job can be processed on only one machine at a time.

The JSSP has traditionally been represented using the disjunctive graph notation, as shown in Fig. 1. There are $N \times M$ operation nodes, and operations that can be processed

on the same machine are connected by disjunctive edges (see Fig. 1(a)), which indicate that their processing order on the machine has not yet been determined. Once a schedule is fixed, these disjunctive edges are converted into conjunctive arcs, forming a path that indicates the operation sequence on each machine, as shown in Fig. 1(b).

To further develop our idea, we introduce the concept of a *task*, which refers to an operation that a machine needs to perform. In a feasible schedule, each machine m_k will process N tasks denoted as $\{\tau_1, \dots, \tau_N\}$. Figure 1(c) shows the alignment of the operation nodes according to their processing order on a specific machine. For example, machine m_1 processes operations in the order o_{12} , o_{22} , o_{13} , which correspond to tasks τ_{11} , τ_{21} , τ_{31} , respectively. This representation provides a clearer view of the task sequence on each machine.

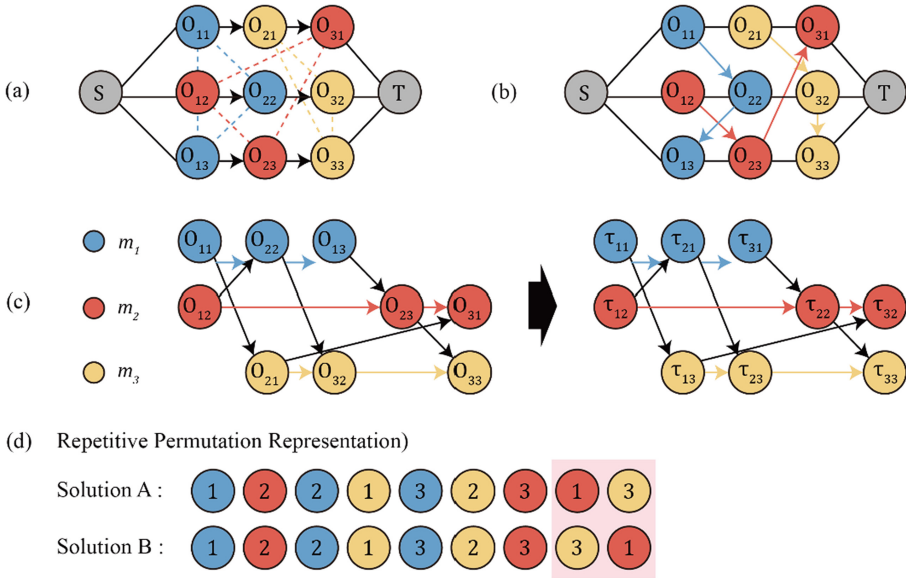


Fig. 1. Graph representation of a JSSP

For any feasible schedule, there exists a one-to-one correspondence between an arbitrary j -th operation of job i , denoted as o_{ji} , and an l -th task on machine m_k . To express this one-to-one mapping, we define an assignment function ϕ as follows:

$$\phi(\tau_{lk}) = (j, i) \quad (1)$$

A projection function π is defined as $\pi : O \rightarrow J \times I$, where $J = \{j | j \in [1, 2, \dots, M]\}$ and $I = \{i | i \in [1, 2, \dots, N]\}$, so that the operation index and job index corresponding to operation o_{ji} can be expressed as:

$$\pi(o_{ji}) = (j, i) \quad (2)$$

In addition, a projection function $\pi_1 : J \times I \rightarrow J$ extracts the operation index j and a projection function $\pi_2 : J \times I \rightarrow I$ extracts the job index i of operation o_{ji} . In terms of tasks, the index j of task τ_{lk} in its corresponding job can be represented as:

$$j = \pi_1(j, i) = \pi_1(\phi(\tau_{lk})) \quad (3)$$

Similarly, the job index i corresponding to task τ_{lk} can be expressed as:

$$i = \pi_2(j, i) = \pi_2(\phi(\tau_{lk})) \quad (4)$$

From the graph-based representation, the starting time s_{ji} of an operation o_{ji} cannot be earlier than the maximum of two values: the completion time of its preceding operation $t_{(j-1),i}$ within the same job, and the completion time of its preceding task $t_{\phi(\tau(l-1),k)}$ on the same machine. Assuming no unnecessary slack time is allowed during the scheduling process, the start time is calculated as Eq. (5). Since the termination time t is determined by adding the processing time p_{ji} of operation o_{ji} to the start time s , we obtain the following recursive relation as in Eq. (6).

$$s_{ji} = \max(t_{(j-1),i}, t_{\phi(\tau(l-1),k)}) \text{ where } (j, i) = \phi(\tau_{lk}) \quad (5)$$

$$t_{ji} = \max(t_{(j-1),i}, t_{\phi(\tau(l-1),k)}) + p_{ji} \quad (6)$$

The objective of the JSSP is to minimize the maximum cycle time of all jobs, also known as the *makespan*.

$$\text{Minimize } c_{\max} = \text{Minimize } \max\{t_{ji} | 1 \leq i \leq M, 1 \leq j \leq N\} \quad (7)$$

Due to the complex constraints embedded in the problem, it is crucial to adopt a solution representation that inherently satisfies the operational constraints. To this end, Gen et al. (1994) and Bierwirth (1995) proposed a method that represents the solution using a repetitive permutation of job indices [5, 10]. In this method, the solution sequence σ includes each job number exactly M times. For any job J_p , the operations are interpreted in the order of their appearance as $o_{1p}, o_{2p}, \dots, o_{Mp}$. Within the job sequence, jobs that appear earlier are scheduled earlier on the corresponding machine. A single solution can be mapped to multiple such job sequences, as illustrated in Fig. 1(d). The nomenclature used in this paper is presented in Table 1.

4 Methodology

4.1 RUBI: Relative-Urgency Based Initialization

We propose a solution construction methodology named RUBI (Relative-Urgency Based Initialization), that can be used as an initial solution in the search process for optimizing the makespan. The core idea is to prioritize the processing of tasks belonging to jobs that have the “longest way to go.” In other words, jobs with more remaining operations are considered relatively more urgent and are thus given priority in the schedule, allowing less urgent jobs to yield their turn.

Table 1. Nomenclature

N	Number of jobs
M	Number of machines
o_{ji}	Operation of job J_i at position j
p	Processing time of operation o_{ji}
s_{ji}	Start time of operation o_{ji}
t_{ji}	Termination time of operation o_{ji}
τ_{lk}	l -th task of machine m_k
S	Start node in the disjunctive graph representation of JSSP
T	Termination node in the disjunctive graph representation of JSSP
$\phi(\cdot)$	Assignment function mapping a task to its corresponding operation index
$\pi(\cdot)$	Projection function returning the operation index and job index as a tuple
$\pi_1(\cdot)$	Projection function extracting the operation index from a tuple
$\pi_2(\cdot)$	Projection function extracting the job index from a tuple
$\mathcal{M}$	Machine-ready set
$\mathcal{J}$	Job-ready set
$\mathcal{O}$	Set of all operations o_{ji}
$\mathcal{R}$	Ready-operation set (intersection of $\mathcal{M}$ and $\mathcal{J}$)
σ	Solution sequence
$c_{\max}$	Makespan; $\max(t_{ji})$

We begin by considering Eq. (1): suppose two different operations o_{1p} and o_{jq} are both waiting to be processed as the first task on machine m_k . To minimize the makespan $\max(t_{ji})$, it is necessary to minimize the termination time t_{ji} of each operation o_{ji} as much as possible. Since there are no prior operations within job J_p that must be processed before o_{1p} , the available starting time s_{1p} is determined solely by the termination time of the preceding task on machine m_k . If no such task exists (i.e., the machine is idle at the beginning), it is initialized to 0.

Now assuming $1 \ll j$, we consider two cases:

- Case 1: Process o_{1p} first, then o_{jq} .
- Case 2: Process o_{jq} first, then o_{1p} .

The termination times for each case are computed as follows:

Case 1:

$$t_{1p} = t_{\phi(\tau_{1,k})} = \max(0, 0) + p_{1p} = p_{1p} \quad (8)$$

$$t_{jq} = t_{\phi(\tau_{2,k})} = \max(t_{j-1,q}, t_{\phi(\tau_{1,k})}) + p_{jq} = \max(t_{j-1,q}, p_{1p}) + p_{jq} \quad (9)$$

Case 2:

$$t_{jq} = t_{\phi(\tau_{1,k})} = \max(t_{j-1,q}, 0) + p_{jq} = t_{j-1,q} + p_{jq} \quad (10)$$

$$t_{1p} = t_{\phi(\tau_{2,k})} = \max(0, t_{\phi(\tau_{1,k})}) + p_{1p} = t_{j-1,q} + p_{jq} + p_{1p} \quad (11)$$

If all processing times are strictly greater than zero and the number of operations M is greater than one, all termination times of Job J_p 's operations must necessarily be greater than t_{1p} . Therefore, the lower bound of the makespan for Job J_p in Case 1 is roughly estimated as p_{1p} , while in Case 2 it becomes $t_{j-1,q} + p_{jq} + p_{1p}$. Our strategy is to choose the scheduling option that leads to the smallest possible lower bound. Accordingly, for each machine m_k , given a set of unfinished operations as task candidates, the operation with the smallest operation index is selected as the next task. This selection rule is referred to as the Relative-Urgency (RU) Principle. The RU principle is a type of restricted formulation that guides the search towards a better feasible solution space. The principle is formally defined as follows:

$$\pi_1(\phi(\tau_{pk})) \leq \pi_1(\phi(\tau_{qk})) \text{ for all pairs } \tau_{pk}, \tau_{qk} \text{ such that } p < q \quad (12)$$

A solution generated under this principle is referred to as an RU solution. The pseudo-code of generating an RU solution is presented in Table 2. First, we define two sets: the machine-ready set $\mathcal{M}$ and the job-ready set $\mathcal{J}$. Operations with operation index equal to 1 are considered top priority from the machine's perspective and are thus automatically included in $\mathcal{M}$. Likewise, since they are also the first operations in their respective jobs, they are included in $\mathcal{J}$.

The intersection of these two sets, i.e., the operations that belong to both $\mathcal{M}$ and $\mathcal{J}$, are defined as the ready-operation set $\mathcal{R}$. At each decision point, one operation from $\mathcal{R}$ is randomly selected and added to the schedule. The next operation in the same job is then automatically added to the job-ready set $\mathcal{J}$. Starting from operations whose operation index is equal to 1 (pivot), the machine-ready set $\mathcal{M}$ is updated by incrementing the pivot by 1 once all operations with the current pivot index have been scheduled. Operations whose operation index is equal to the updated pivot are then added to $\mathcal{M}$. Note that when multiple operations in $\mathcal{R}$ exhibit the same priority on their compatible machines, one of them is selected at random. This random selection allows the generation of different solutions with each execution, thereby facilitating the construction of a diverse population of solutions.

The procedure of generating RU solutions of a 3×3 JSSP is illustrated in Fig. 2. The algorithm begins with three ready-operations waiting to be added to the schedule. Once operation o_{12} is selected, the following operation in the same job, o_{22} , is added to the job-ready set. However, o_{22} must wait until its designated machine has completed all other operations with smaller operation indices. In this case, operations o_{11} and o_{13} must be processed first. Only after these operations are scheduled can o_{22} be included in the machine-ready set.

Our research hypothesis is that utilizing the RU solutions in the initialization process of optimization algorithms can enhance the search efficiency of the enumerative process. In the following section, the efficiency of RUBI will be evaluated in greater detail.

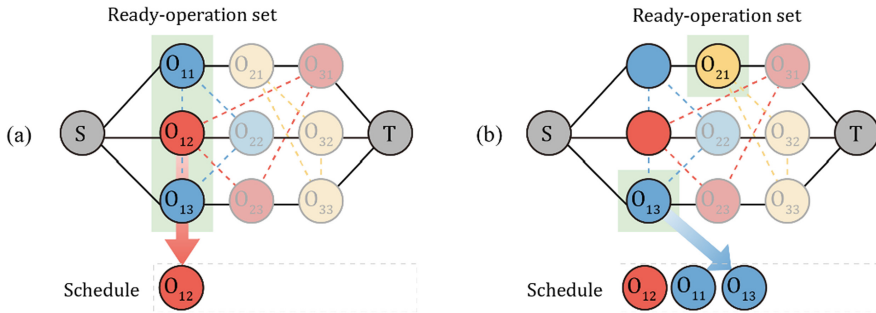
Table 2. Pseudo-code of Generating an RU solution (RUBI)**Algorithm 1** RUBI (Relative-Urgency Based Initialization)

Input: Number of jobs N , number of machines M , set of all operations $\mathcal{O}$
Output: A feasible solution σ satisfying job precedence and RU principle

```

1  Initialize  $\text{pivot}[k] \leftarrow 1$  for all  $m_k$ ; Initialize solution sequence  $\sigma \leftarrow \emptyset$ ;
   Initialize  $\mathcal{M} \leftarrow$  operations  $o_{ji}$  such that  $j = \text{pivot}[k]$  and  $o_{ji}$  requires  $m_k$ ;
   Initialize  $\mathcal{J} \leftarrow$  operations  $o_{ji}$  such that  $j = 1$  (first operation of each job);
2  while there exist unscheduled operations do
3       $\mathcal{R} \leftarrow \mathcal{M} \cap \mathcal{J}$ ; // Ready-operation set
4      Choose  $o_{ji} \in \mathcal{R}$ 
5      Append  $o_{ji}$  to  $\sigma$ 
6      Remove  $o_{ji}$  from  $\mathcal{M}$  and  $\mathcal{J}$ 
7      if next operation  $o_{(j+1)i}$  exists then
8          | Add  $o_{(j+1)i}$  to  $\mathcal{J}$ 
9      end
10     Foreach machine  $m_k$  do
11         if  $\mathcal{M}$  has no operation requiring  $m_k$  then
12             | Increment  $\text{pivot}[k] \leftarrow \text{pivot}[k] + 1$ 
13             | Add to  $\mathcal{M}$  all unscheduled operations  $o_{ji}$  such that  $j = \text{pivot}[k]$ 
13             | and  $o_{ji}$  requires  $m_k$ 
14         End
15     End
16 End
17 return  $\sigma$ 

```

**Fig. 2.** Procedure of generating an RU solution**4.2 Efficiency Analysis of RUBI**

For a partial solution sequence σ of length N' , the selection of the $(N' + 1)$ -th element determines the subset of the solution space to which the sequence will eventually belong. Suppose the expected makespan for the partial solution generated by following the RU principle is l_a , while the lower bound for a different is l_b , and $l_a \leq l_b$. In such a case, if

the true optimal lies between l_a and l_b , the decision made according to the RU principle is beneficial for guiding the search toward the optimal region. However, this does not necessarily guarantee that the optimal solution will be found through the RU principle. If the optimal makespan is greater than l_b , then the subset of solutions selected by following the RU principle may or may not contain the optimal solution.

To demonstrate the usefulness of the RU principle, we define a metric called the RU score, which quantifies how well a given feasible solution reflects the RU principle. The RU score is computed as the L1 distance between the task order vector of a machine in a solution sequence and its sorted version. For example, if the task order of machine m_k is $\{\pi_l(\phi(\tau_{lk})) | l = 1, 2, \dots, N\} = \{1, 2, 4, 3\}$, then its sorted version is $\{1, 2, 3, 4\}$. The RU score is the L1 distance between them, which equals 2. Solutions that partially follow the RU principle can be easily generated by proportionately splitting an RU solution sequence into two parts and reshuffling a slice. This allows us to generate solutions that reflect the RU principle to varying degrees. Using these partially-RU solutions, along with RU solutions and randomly generated ones, we calculate the correlation between the RU score and the makespan.

5 Experimental Results

5.1 Correlation Between RU Score and Makespan

To validate the research hypothesis, JSSP instances of size 10×10 , 20×20 , 50×50 , and 100×100 were generated. Upon these instances, three types of solution groups were created, each labeled as Group A, B, and C respectively. Group A consists of randomly generated solutions; Group B contains solutions that partially incorporate the RU principle; and Group C consists of RU solutions. For each group, 100 individual solutions were generated. After calculating the makespan for each solution, a strong correlation between the RU score and the makespan was observed, as shown in Fig. 3.

When projected into the two-dimensional space spanned by the makespan and the RU score, the solutions in Group C appeared near the region with lower makespan, while the solutions in Group A were located far from the optimal region. Partially-RU solutions (Group B) filled the space between Groups A and C.

From this observation, it can be inferred that applying the RU principle allows the search process to begin in a region with lower makespan, compared to randomly generated solutions. The correlation was more clearly observed in larger JSSP instances than in smaller case. This highlights the significance of RUBI as a potential breakthrough for the curse of dimensionality, which means that the random initialization becomes ineffective as the problem size increases.

5.2 Evaluating RUBI on Benchmark Datasets

To validate the efficiency of RUBI, we applied it within a JSSP search framework using a GA. In GA, solutions are encoded as chromosomes, and a set of such solutions, known as the population, evolves by the principle of natural selection. Solutions with better objective function values are more likely to survive in the next iteration, or generation.

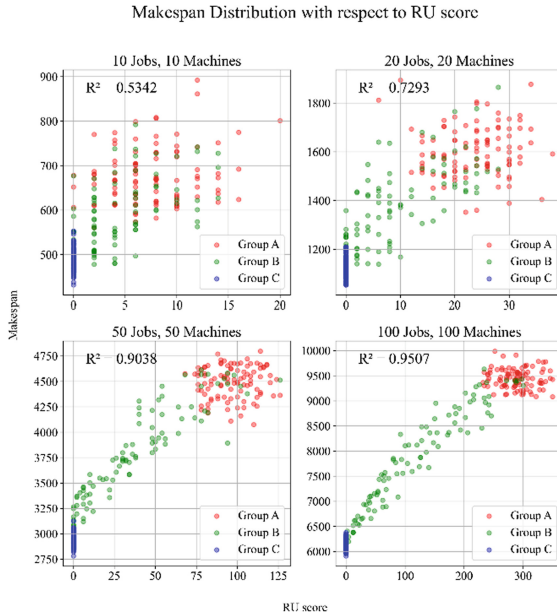


Fig. 3. Correlation analysis between RU score and makespan

These solutions may either persist or become parents to reproduce offsprings, which are newly introduced solutions in the population.

The reproduction process consists of two main operations: crossover and mutation. The former combines two parent solutions with desirable characteristics into a new offspring solution, while the latter introduces random changes to a solution. Because of these modification mechanisms, GA is considered to have the ability to overcome premature convergence caused by local optima and selected as the validation method for evaluating the effectiveness of RUBI in this study. Random initialization, two different heuristics (SPT and LPT), and GT algorithm were selected for comparison. As benchmark test instances, the well-known LA instances provided by Lawrence (1984) were used.

To ensure reproducibility, all experiments were repeated five times. The crossover and mutation probabilities were set to 0.7 and 1.0, respectively. The Partially Mapped Crossover (PMX) operator was employed for crossover, and a composite mutation operator was used for mutation, in which either a swift mutation or a swap mutation was randomly selected during the mutation phase. The selection method adopted in this study, referred to as seed selection, chooses the individual with the best fitness with 50% probability, and with the remaining 50%, it selects a different individual randomly.

The experimental results are summarized in Table 3. The best observed makespan values across five runs are recorded, with the average execution time in seconds only in the cases where the optimal solution was found in at least one of the five runs. In some cases, the 5 methods successfully discovered the optimal schedule. However, in certain

problems—particularly those with larger problem sizes—all 5 initialization techniques were not able to reach the optimal solution, likely due to convergence to local minima.

Table 3. Best makespan obtained by 5 initialization methodologies upon LA instances

Instance	Optimal	RUBI	Random	SPT	LPT	GT
la01	666	666 (169.87)	666 (121.7)	666 (61.61)	666 (154.25)	666 (85.77)
la02	655	659 -	655 (298.38)	680 -	676 -	667 -
la03	597	617 -	617 -	638 -	637 -	645 -
la04	590	598 -	598 -	598 -	612 -	607 -
la05	593	593 (0.34)	593 (13.78)	593 (0.31)	593 (7.057)	593 (22.38)
la06	926	926 (15.99)	926 (120.83)	926 (65.17)	926 (68.15)	926 (54.18)
la07	890	890 (348.80)	890 (372.54)	897 -	905 -	890 (378.94)
la08	863	863 (131.90)	863 (175.49)	863 (218.74)	863 (268.44)	863 (171.31)
la09	951	951 (2.82)	951 (120.51)	951 (23.29)	951 (72.45)	951 (90.42)
la10	958	958 (0.42)	958 (55.07)	958 (52.60)	958 (88.34)	958 (54.71)
la11	1222	1222 (36.97)	1222 (201.13)	1222 (409.09)	1222 (214.62)	1222 (202.55)
la12	1039	1039 (22.27)	1039 (197.60)	1039 (255.73)	1039 (291.8)	1039 (251.40)
la13	1150	1150 (6.027)	1150 (210.24)	1150 (113.80)	1150 (385.53)	1150 (164.77)
la14	1292	1292 (0.59)	1292 (38.96)	1292 (302.22)	1292 (63.20)	1292 (136.82)
la15	1207	1207 (498.3)	1207 (824.82)	1249 -	1243 -	1244 -
la16	945	982 -	1005 -	1003 -	1010 -	979 -

(continued)

Table 3. (continued)

Instance	Optimal	RUBI	Random	SPT	LPT	GT
la17	784	793	805	854	806	833
		-	-	-	-	-
la18	848	865	861	942	951	904
		-	-	-	-	-
la19	842	896	888	877	909	902
		-	-	-	-	-
la20	902	936	955	929	969	919
		-	-	-	-	-



Fig. 4. Comparison of initialization methods by execution time

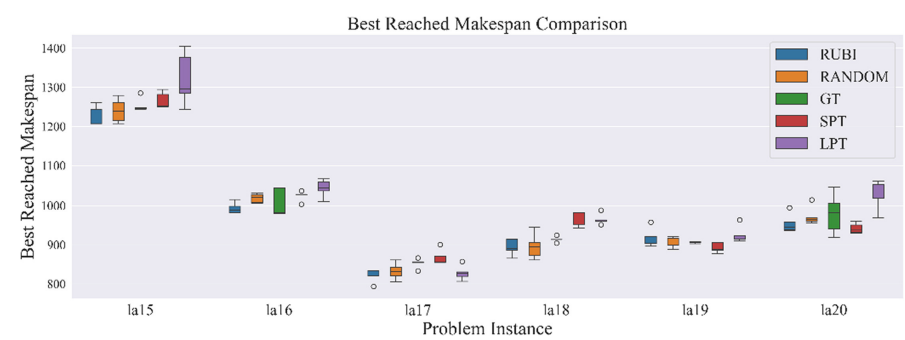


Fig. 5. Comparison of initialization methods by best reached makespan

Despite this, a clear distinction was observed in the time required to reach the optimal solution, as illustrated in Fig. 4. Among the cases where the optimal solution was found by all 5 initialization methods, RUBI achieved the optimal solution in significantly less time in 9 out of 10 problems, sometimes requiring less than one second.

Moreover, in 6 problem instances where the optimum is not found, RUBI still demonstrated lower makespan when compared with other methodologies, as shown in Fig. 5. The results suggest that RUBI allows the enumerative algorithm to navigate more effectively toward regions of the search space where optimal solutions are likely to be found.

6 Conclusion

This study presents a new initialization scheme for constructing solution sequences in the JSSP. The proposed method, named Relative-Urgency Based Initialization (RUBI), is a heuristic that is constraint-aware and explicitly incorporates problem structure at the initialization level. RUBI is grounded in the principle of prioritizing jobs with more remaining operations—termed as "relatively urgent" jobs. Such decisions, when applied during the sequential construction of a solution, can contribute to lowering the expected makespan of partial solutions, increasing the likelihood of reaching the minimum of the final makespan.

To validate the efficiency of RUBI as an initialization strategy, the relationship between the RU score—which quantifies how well a solution reflects the RU principle—and the makespan was analyzed across four different JSSP instances. A strong positive correlation observed, and the correlation coefficients exceeded 0.9 especially in problems with larger size. This supports the hypothesis that solutions generated using RUBI are structurally closer to the optimal region compared to randomly initialized solutions.

Furthermore, in benchmark tests using the LA instances, RUBI-GA achieved the optimal solution significantly faster than random, SPT, LPT and GT initialization heuristics in 9 out of 10 cases, sometimes requiring less than one second. Even when the problem size was so large that the optimum could not be achieved, RUBI demonstrated the best performance among the 5 initialization schemes. These findings demonstrate RUBI's effectiveness in reducing the computational burden while improving the likelihood of reaching optimal or near-optimal solutions.

The contribution of this work can be summarized in two ways; First, RUBI enables the search technique to converge more quickly to high-quality solutions. Second, since RUBI is formulated based on structural characteristics that are independent of problem size, it is especially effective for large-scale JSSP instances where traditional methods often fail due to the curse of dimensionality. Future research will explore the integration of RUBI with hybrid metaheuristics, such as local-search-enhanced genetic algorithms, to further improve performance. Further application to other scheduling problems, including the Flexible Job Shop Scheduling Problem (FJSSP), and to real-world applications such as vehicle routing and manufacturing system optimization is also expected.

Acknowledgments. This research was supported by (a) the Ministry of Trade, Industry and Energy of the Republic of Korea (Project No. P0022929), (b) the National Research Foundation of Korea (NRF) grant funded by the Korean government (Grant No. RS-2025-00555741), and (c) the Research Institute of Marine Systems Engineering at Seoul National University.

Disclosure of Interests. The authors have no conflicts of interest to declare.

References

1. Abdolrazzaghi-Nezhad, M., Abdullah, S.: A robust intelligent construction procedure for job-shop scheduling. *Inform. Technol. Control* **43**(3), 217–229 (2014)
2. Adams, J., Balas, E., Zawack, D.: The shifting bottleneck procedure for job shop scheduling. *Manage. Sci.* **34**(3), 391–401 (1988)
3. Alkhateeb, F., Abed-alguni, B.H., Al-roushan, M.H.: Discrete hybrid cuckoo search and simulated annealing algorithm for solving the job shop scheduling problem. *J. Supercomput.* **78**(4), 4799–4826 (2022)
4. Asadzadeh, L.: A parallel artificial bee colony algorithm for the job shop scheduling problem with a dynamic migration strategy. *Comput. Ind. Eng.* **102**, 359–367 (2016)
5. Bierwirth, C.: A generalized permutation approach to job shop scheduling with genetic algorithms. *Oper.-Res.-Spektrum* **17**(2), 87–92 (1995)
6. Brucker, P., Jurisch, B.: A new lower bound for the job-shop scheduling problem. *Eur. J. Oper. Res.* **64**(2), 156–167 (1993)
7. Canbolat, Y.B., Gundogar, E.: Fuzzy priority rule for job shop scheduling. *J. Intell. Manuf.* **15**, 527–533 (2004)
8. Conway, R., Maxwell, W., Oldziey, J.: Sequencing Against Due-Dates. Cornell University Operations Research and Industrial Engineering, Technical report (1966)
9. Garrido, A., Salido, M.A., Barber, F., López, M.: Heuristic methods for solving job-shop scheduling problems. In: *Proc. ECAI-2000 Workshop on New Results in Planning, Scheduling and Design (PuK2000)*, pp. 44–49 (2000)
10. Gen, M., Tsujimura, Y., Kubota, E.: Solving job-shop scheduling problems by genetic algorithm. In: *Proceedings of IEEE International Conference on Systems, Man and Cybernetics*, vol. 2, pp. 1577–1582. IEEE (1994)
11. Giffler, B., Thompson, G.L.: Algorithms for solving production-scheduling problems. *Oper. Res.* **8**(4), 487–503 (1960)
12. Gonçalves, J.F., de Magalhães Mendes, J.J., Resende, M.G.: A hybrid genetic algorithm for the job shop scheduling problem. *Eur. J. Oper. Res.* **167**(1), 77–95 (2005)
13. Gong, G., Deng, Q., Chiong, R., Gong, X., Huang, H.: An effective memetic algorithm for multi-objective job-shop scheduling. *Knowl.-Based Syst.* **182**, 104840 (2019)
14. Hoitomt, D.J., Luh, P.B., Pattipati, K.R.: A practical approach to job-shop scheduling problems. *IEEE Trans. Robot. Autom.* **9**(1), 1–13 (1993)
15. Jain, A.S., Meeran, S.: Job-shop scheduling using neural networks. *Int. J. Prod. Res.* **36**(5), 1249–1272 (1998)
16. Jiang, T., Zhang, C.: Application of grey wolf optimization for solving combinatorial problems: job shop and flexible job shop scheduling cases. *IEEE Access* **6**, 26231–26240 (2018)
17. Kuczapski, A.M., Micea, M.V., Maniu, L.A., Cretu, V.I.: Efficient generation of near optimal initial populations to enhance genetic algorithms for job-shop scheduling. *Inform. Technol. Control* **39**(1) (2010)
18. Lourenço, H.R.: Job-shop scheduling: Computational study of local search and large-step optimization methods. *Eur. J. Oper. Res.* **83**(2), 347–364 (1995)
19. Manne, A.S.: On the job-shop scheduling problem. *Oper. Res.* **8**(2), 219–223 (1960)
20. Martin, P., Shmoys, D.B.: A new approach to computing optimal schedules for the job-shop scheduling problem. In: Cunningham, W.H., McCormick, S.T., Queyranne, M. (eds.) *Integer Programming and Combinatorial Optimization. IPCO 1996. Lecture Notes in Computer Science*, vol. 1084. Springer, Heidelberg (1996). https://doi.org/10.1007/3-540-61310-2_29
21. Mattfeld, D.C., Bierwirth, C.: An efficient genetic algorithm for job shop scheduling with tardiness objectives. *Eur. J. Oper. Res.* **155**(3), 616–630 (2004)

22. Nakano, R., Yamada, T.: Conventional genetic algorithm for job shop problems. In: ICGA, vol. 91, pp. 474–479 (1991)
23. Park, B.J., Choi, H.R., Kim, H.S.: A hybrid genetic algorithm for the job shop scheduling problems. *Comput. Ind. Eng.* **45**(4), 597–613 (2003)
24. Peng, C., Wu, G., Liao, T.W., Wang, H.: Research on multi-agent genetic algorithm based on tabu search for the job shop scheduling problem. *PLoS ONE* **14**(9), e0223182 (2019)
25. Pinedo, M., Singer, M.: A shifting bottleneck heuristic for minimizing the total weighted tardiness in a job shop. *Naval Res. Log. (NRL)* **46**(1), 1–17 (1999)
26. Someya, H., Yamamura, M.: A genetic algorithm without parameters tuning and its application on the floorplan design problem. In: *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation*, vol. 1, pp. 620–627 (1999)
27. Tamilarasi, A., et al.: An enhanced genetic algorithm with simulated annealing for job-shop scheduling. *Int. J. Eng. Sci. Technol.* **2**(1), 144–151 (2010)
28. Van Laarhoven, P.J., Aarts, E.H., Lenstra, J.K.: Job shop scheduling by simulated annealing. *Oper. Res.* **40**(1), 113–125 (1992)
29. Wagner, H.M.: An integer linear-programming model for machine scheduling. *Naval Res. Log. Q.* **6**(2), 131–140 (1959)
30. Wang, S., Yu, J.: An effective heuristic for flexible job-shop scheduling problem with maintenance activities. *Comput. Ind. Eng.* **59**(3), 436–447 (2010)
31. Watanabe, M., Ida, K., Gen, M.: A genetic algorithm with modified crossover operator and search area adaptation for the job-shop scheduling problem. *Comput. Ind. Eng.* **48**(4), 743–752 (2005)
32. Yahyaoui, A., Fnaiech, N., Fnaiech, F.: A suitable initialization procedure for speeding a neural network job-shop scheduling. *IEEE Trans. Industr. Electron.* **58**(3), 1052–1060 (2010)
33. Zhang, C., Li, P., Rao, Y., Li, S.: A new hybrid GA/SA algorithm for the job shop scheduling problem. In: Raidl, G.R., Gottlieb, J. (eds.) *Evolutionary Computation in Combinatorial Optimization. EvoCOP 2005. Lecture Notes in Computer Science*, vol. 3448. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-31996-2_23
34. Zhang, X., Zhu, G.Y.: A literature review of reinforcement learning methods applied to job-shop scheduling problems. *Comput. Oper. Res.* 106929 (2024)
35. Zhou, H., Feng, Y., Han, L.: The hybrid heuristic genetic algorithm for job shop scheduling. *Comput. Ind. Eng.* **40**(3), 191–200 (2001)